

CLOUDSHIELD AI

MASTER IMPLEMENTATION BREAKDOWN

12-phase, dependency-ordered execution plan

Version 1.0 • 10 September 2026

1. Purpose

This document decomposes the approved CloudShield AI implementation roadmap into executable engineering work. It sits below the completed specifications and Master Implementation Roadmap and above individual tickets/code changes. It is an execution map, not a replacement for the architecture, requirements, DAIL, testing, evidence, deployment or Definition of Done specifications.

CloudShield AI is the broader product; DAIL is the narrower Terraform-specific research core. The first implementation objective is a deterministic, auditable DAIL mechanism. Product expansion must not weaken the research boundary.

2. Non-Negotiable Rules

- LLM proposes; deterministic systems validate; governance decides; controlled cloud execution occurs only through authorized promotion.
- Candidate state is untrusted until successful promotion; rejected candidates never become the trusted base.
- Only the Promotion Controller may update trusted state.
- Protected invariants require current PASS evidence for the relevant state; invalidation does not itself mean violation.
- UNKNOWN, ERROR, UNSUPPORTED and UNCERTAIN never silently become PASS.
- Identity/dependency uncertainty is conservative.
- Created in-scope resources receive applicable baseline security checks.
- Historical evidence is immutable; every research-critical transition is traceable and replayable.
- Deterministic fixed-patch DAIL must work before live LLM integration.
- Independent ground truth must exist before main experimental claims.
- Changes to research contribution, invariant semantics, experiment baselines or claimed validity require audit/research review.

3. Global Phase Map

Phase	Name	Depends on	Primary outcome
P1	Project & Engineering Foundation	—	Reproducible repo/tooling/CI/Terraform baseline
P2	Platform & Domain Foundation	P1	Tenancy, authz, persistence, lifecycle, jobs, observability
P3	AWS Cloud Integration	P2	Controlled AWS provider boundary
P4	Asset Discovery & Inventory	P3	Canonical supported inventory + identity
P5	Security & Risk Intelligence	P4	Deterministic findings/risk
P6	Dependency & Impact Intelligence	P4/P5	Dependency graph + affected-property analysis
P7	AI Intelligence & Safety	P2/P5/P6	Safe provider-agnostic AI proposal pipeline
P8	Remediation & DAIL Verification	P6/P7*	Candidate → impact → verification → decision

P9	Governance & Cloud Promotion	P3/P8	Controlled AWS mutation + reconciliation
P10	Trusted State & Continuous Control	P4/P5/P6/P9	Drift/invalidation/reverification
P11	Product Experience & Integrations	P8/P9/P10	Operable end-to-end product
P12	Production Hardening & MVP Release	P1–P11	Release-ready validated MVP

* P8 begins with fixed patches; the live LLM is not a prerequisite for the deterministic DAIL core.

4. Milestone Gates

Milestone	Phases	Gate objective
M1 Foundation	P1–P2	Safe, reproducible engineering/platform base
M2 Cloud Intelligence	P3–P6	Deterministic AWS/Terraform understanding and impact
M3 AI + Verification	P7–P8	Safe AI proposal plus validated DAIL control loop
M4 Governed Autonomy	P9–P10	Controlled promotion and continuously reconciled trust
M5 Product + Production	P11–P12	Usable, hardened, recoverable MVP

P1 — Project & Engineering Foundation

Objective: Establish the repository, stack, local environment, quality system, Terraform structure and engineering governance needed for reproducible implementation.

Entry: Approved specifications and implementation baseline.

Workstreams

ID	Workstream	Scope
P1.1	Repository	Monorepo boundaries for backend, frontend, shared contracts, workers, DAIL, tests, infra and docs.
P1.2	Backend	Application bootstrap, configuration, DI, health, errors.
P1.3	Frontend	Shell, routing, API boundary, auth-session placeholder.
P1.4	Tooling	Python/uv, formatting, linting, typing, pytest, scripts.
P1.5	CI	PR checks, tests, security scans, artifacts.
P1.6	Terraform	Environment/module structure, validation and state conventions.
P1.7	Governance	ADRs, decision log, milestone reporting, change control.

Milestones

ID	Outcome	Evidence
P1-M1	Clean checkout	Install/start/test succeeds from documented commands.
P1-M2	Skeleton	Backend/frontend/shared packages build and smoke-test.
P1-M3	CI/Terraform	Automated quality checks and Terraform validation work.
P1-G	Foundation gate	Reproducible local state with no unintended authority.

Implementation Tasks

P1.1 — Create repository structure

- Keep DAIL explicit as a subsystem/module boundary.

- Separate domain logic from transport and provider SDKs.

P1.2 — Lock practical stack baseline

- Python 3.12, uv, Pydantic v2, SQLAlchemy, pytest, boto3, NetworkX, Terraform CLI, GitHub Actions are the recommended baseline.
- Record alternatives/rationale in ADRs.

P1.3 — Create local environment

- Document one clean-checkout bootstrap.
- Prepare deterministic Terraform fixtures.

P1.4 — Create quality gates

- Format/lint/type/unit tests.
- Dependency and secret scanning.

P1.5 — Create infrastructure skeleton

- Terraform directories, environments and naming/tagging conventions.
- No production mutation authority yet.

Tests

- Clean-checkout test
- Backend smoke test
- Frontend build test
- Shared contract serialization test
- Terraform fmt/validate test
- CI failure-path test
- Secret/dependency scan

Exit Gate

Pass only when a new developer can reproduce the baseline and automated quality checks run without granting production or promotion authority.

Deliverables

- Repository
- Local dev setup
- CI
- Terraform skeleton
- ADRs
- Bootstrap documentation

Risks / Escalations

- Over-engineering
- Undocumented machine dependencies
- DAIL boundary blurred

P2 — Platform & Domain Foundation

Objective: Implement the shared platform primitives: tenancy, authorization, persistence, lifecycle, jobs, concurrency, idempotency and observability.

Entry: P1 gate passed.

Workstreams

ID	Workstream	Scope
P2.1	Tenant & Scope	Organization/tenant/project/cloud scope ownership.
P2.2	Identity & Authz	Roles, permissions and server-side authorization.
P2.3	Persistence	PostgreSQL/SQLAlchemy, migrations, repositories.
P2.4	Lifecycle	Legal transitions, versioning, optimistic concurrency.
P2.5	Jobs & Events	Durable jobs, retries, idempotency, event contracts.
P2.6	Observability	Logs, metrics, traces, audit/evidence interfaces.

Milestones

ID	Outcome	Evidence
P2-M1	Persistence	Core entities persist with tenant isolation.
P2-M2	Lifecycle	Illegal transitions are rejected.
P2-M3	Async	Accepted jobs are durable/idempotent.
P2-G	Platform gate	Authorized state change is durable, observable and auditable.

Implementation Tasks

P2.1 — Build tenancy

- Every scoped domain object has explicit ownership.
- Authorization is server-side.

P2.2 — Build authz

- Central authorization service/policy boundary.
- No UI-only permission checks.

P2.3 — Build persistence

- Migrations + repositories + transaction boundaries.
- Historical evidence remains versioned.

P2.4 — Build lifecycle primitives

- Centralize legal transitions.
- Add version/concurrency checks.

P2.5 — Build jobs

- Persist before acknowledgement.
- Define retry, timeout and reconciliation states.

P2.6 — Build telemetry

- Keep logs/metrics/traces/events/audit/evidence distinct.
- Redact secrets.

Tests

- Tenant-isolation tests
- Authorization bypass tests
- Migration tests
- Illegal-transition tests
- Concurrent update/idempotency tests
- Job retry tests

- Audit/evidence durability tests

Exit Gate

Pass only when authorized tenant-scoped operations can be persisted and transitioned safely with observable/auditable behavior.

Deliverables

- Core schema
- Authz boundary
- Lifecycle engine
- Job abstraction
- Observability foundation

Risks / Escalations

- Schema lock-in too early
- Duplicated authorization
- String-only state machines

P3 — AWS Cloud Integration

Objective: Create the provider abstraction and controlled AWS read boundary, including STS/IAM, retries and deterministic mocks.

Entry: P2 gate passed.

Workstreams

ID	Workstream	Scope
P3.1	Provider Interface	Provider-neutral connection/discovery/observation/mutation contracts.
P3.2	AWS Adapter	boto3 implementation for bounded MVP services.
P3.3	Connection	Role assumption, identity verification, connection metadata.
P3.4	Reliability	Timeout, throttling, pagination, retries, error normalization.
P3.5	Credential Boundary	Short-lived/least-privilege credentials isolated from AI/domain.
P3.6	Mock Provider	Deterministic fixture provider and failure injection.

Milestones

ID	Outcome	Evidence
P3-M1	AWS connection	Controlled account identity verified.
P3-M2	Adapter boundary	Domain has no direct boto3 dependency.
P3-M3	Reliability	Transient AWS failures behave predictably.
P3-G	Cloud gate	Safe controlled AWS read integration works.

Implementation Tasks

P3.1 — Define CloudProvider interfaces

- Separate capabilities for read/discovery and later mutation.
- Expose provider capabilities explicitly.

P3.2 — Implement AWS adapter

- Bound MVP: VPC, subnet, route table, association, IGW, EC2, security groups/rules and RDS metadata.

P3.3 — Implement STS/IAM

- Prefer role assumption and least privilege.
- Never place credentials in prompts/logs.

P3.4 — Centralize AWS calls

- Retry only safe/transient failures.
- Normalize provider errors and preserve provenance.

P3.5 — Implement mock provider

- Deterministic responses and injected throttling/failure cases.

Tests

- Provider contract tests
- AWS identity tests
- Pagination tests
- Throttle/retry tests
- Credential-redaction tests
- Least-privilege review

Exit Gate

Pass only when controlled AWS access works through an adapter with no unsafe credential or promotion path.

Deliverables

- AWS adapter
- Connection API
- IAM design
- Mock provider
- Error/retry layer

Risks / Escalations

- Overbroad IAM
- SDK leakage into domain
- Unsafe retry semantics

P4 — Asset Discovery & Inventory

Objective: Build canonical supported-resource identity, discovery, normalization, snapshots, diffs and freshness semantics.

Entry: P3 gate passed.

Workstreams

ID	Workstream	Scope
P4.1	Resource Identity	Canonical keys and provider identifiers.
P4.2	Discovery	Enumerate supported resources with checkpoints.
P4.3	Normalization	Provider →canonical resource representation.
P4.4	Snapshots/Diffs	Immutable observations and before/after change sets.
P4.5	Reconciliation	Created/deleted/replaced/modified/uncertain cases.
P4.6	Freshness	Observed time, provenance, confidence, stale/partial/unknown.

Milestones

ID	Outcome	Evidence
P4-M1	Identity	Supported resources classify deterministically.
P4-M2	Inventory	Controlled fixture yields canonical inventory.
P4-M3	Diff	Before/after state yields expected changes.
P4-G	Inventory gate	Inventory is reproducible and conservative.

Implementation Tasks

P4.1 — Implement identity engine

- Include account/region/type/provider identifier.
- Define replacement/recreation behavior.

P4.2 — Implement discovery jobs

- Pagination/checkpoints/partial progress.
- Never label incomplete discovery complete.

P4.3 — Normalize resources

- Preserve provenance and stable canonical representation.

P4.4 — Implement snapshots/diffs

- Historical snapshots immutable.
- Compare relevant attributes and resources.

P4.5 — Implement reconciliation

- Conservative handling of restructuring and uncertainty.

P4.6 — Expose inventory API

- Filtering, pagination, freshness and scope enforcement.

Tests

- Golden inventory tests
- Identity restructuring tests
- Created/deleted/replaced/modified tests
- Partial/unknown/stale tests
- Checkpoint/retry tests

Exit Gate

Pass only when controlled fixtures produce deterministic inventory and ambiguous identity/state is visible and conservative.

Deliverables

- Identity engine
- Discovery worker
- Snapshot/diff engine
- Inventory API
- Golden fixtures

Risks / Escalations

- Provider IDs vs Terraform identity
- Overclaiming completeness
- Overwriting history

P5 — Security & Risk Intelligence

Objective: Implement deterministic rules, findings, evidence and risk scoring without relying on AI for correctness.

Entry: P4 gate passed.

Workstreams

ID	Workstream	Scope
P5.1	Rule Registry	Versioned MVP security/functional predicates.
P5.2	Rule Engine	Deterministic evaluation and evidence.
P5.3	Findings	Lifecycle, severity, affected resources and evidence.
P5.4	Risk	Deterministic scoring with traceable inputs.
P5.5	Policy	Blocking/non-blocking policy semantics.
P5.6	API	Finding/risk retrieval and filtering.

Milestones

ID	Outcome	Evidence
P5-M1	Rules	Canonical scenarios evaluate deterministically.
P5-M2	Findings	Findings have evidence-backed lifecycle.
P5-M3	Risk	Scores are reproducible.
P5-G	Security gate	Known security cases are detected without AI.

Implementation Tasks

P5.1 — Define MVP predicates

- Version rules.
- Keep generic findings distinct from protected DAIL invariants where appropriate.

P5.2 — Implement evaluator

- PASS/FAIL/UNKNOWN semantics.
- Missing data cannot imply PASS.

P5.3 — Implement findings

- Store rule/version/state/affected resources/evidence.

P5.4 — Implement risk

- Record scoring inputs.
- Avoid opaque model dependence.

P5.5 — Implement policy

- Policy can block but cannot mutate trusted state.

Tests

- Rule unit tests
- Evidence-link tests
- Unknown/partial tests
- Finding lifecycle tests
- Risk reproducibility tests

Exit Gate

Pass when supported configurations yield deterministic, evidence-backed findings/risk.

Deliverables

- Rule registry
- Rule engine
- Finding/risk models
- Policy engine
- Security fixtures

Risks / Escalations

- Unsupported AWS semantics
- Subjective risk scoring
- CSPM logic conflated with DAIL

P6 — Dependency & Impact Intelligence

Objective: Build the supported dependency graph, change analyzer and impact/invalidation engine that maps candidate transitions to affected protected obligations.

Entry: P4 and P5 gates passed.

Workstreams

ID	Workstream	Scope
P6.1	Dependency Model	Typed edges + provenance/confidence.
P6.2	Graph Builder	Supported Terraform/infrastructure/semantic relationships.
P6.3	Graph Queries	Direct/transitive dependency queries.
P6.4	Change Analyzer	Resource/attribute transition classification.
P6.5	Impact Engine	Changed resources → affected obligations.
P6.6	Invalidation	Invalidate stale evidence without declaring violation.

Milestones

ID	Outcome	Evidence
P6-M1	Graph	Supported fixture graph is deterministic.
P6-M2	Changes	Candidate transition produces expected change set.
P6-M3	Impact	Affected obligations match scenarios.
P6-G	Impact gate	Canonical regression impact is identified.

Implementation Tasks

P6.1 — Define edge taxonomy

- Resource/reference/network/semantic relationships.
- Attach confidence and provenance.

P6.2 — Build graph

- Bounded MVP semantics; NetworkX is the recommended initial library.
- Do not claim complete AWS semantics.

P6.3 — Build graph queries

- Direct/transitive queries with deterministic output.

P6.4 — Build change analyzer

- Compare trusted vs candidate state.
- Classify create/delete/modify/replace.

P6.5 — Build impact mapping

- Distinguish AFFECTED from VIOLATED.
- Map changes to protected properties.

P6.6 — Build invalidation

- Historical evidence remains immutable but becomes insufficient when applicability changes.

Tests

- Graph determinism
- Restructuring identity
- Direct/transitive dependency
- Impact scenario
- Invalidation semantics
- Unknown dependency tests

Exit Gate

Pass when the canonical regression candidate identifies the affected protected obligation and uncertainty is preserved.

Deliverables

- Dependency graph
- Change analyzer
- Impact/invalidation engine
- Graph/impact fixtures

Risks / Escalations

- Graph overclaim
- False confidence from identity uncertainty
- Affected incorrectly treated as violated

P7 — AI Intelligence & Safety

Objective: Connect a provider-agnostic LLM layer only after deterministic context and DAIL boundaries exist.

Entry: P2 plus usable P5/P6 contracts.

Workstreams

ID	Workstream	Scope
P7.1	AI Abstraction	Gateway/provider registry/adapters contracts.
P7.2	Context	Authorized, minimal, versioned context packages.
P7.3	Prompting	Versioned task prompts and output schemas.
P7.4	Structured Output	Bounded Terraform patch proposal contract.
P7.5	Guardrails	Injection defense, validation, data minimization, tool restrictions.
P7.6	Reliability	Timeouts, retry, routing, quotas, cost/latency telemetry.
P7.7	Evaluation Hooks	Model/config/prompt/version metadata.

Milestones

ID	Outcome	Evidence
P7-M1	Gateway	Domain is vendor-neutral.
P7-M2	Context	Only authorized relevant context is supplied.
P7-M3	Proposal	Malformed/unsafe outputs are rejected.
P7-G	AI gate	AI has proposal capability but zero trusted-state authority.

Implementation Tasks

P7.1 — Build LLM gateway

- Mock adapter first.
- Domain never imports vendor SDK.

P7.2 — Build context assembler

- Trusted state/version + finding + relevant impact/dependencies + constraints.
- Redact secrets.

P7.3 — Version prompts/schemas

- Record model/provider/prompt/config version per run.

P7.4 — Validate proposals

- Reject unsupported resources/operations.
- Only DAIL Candidate State Builder creates candidate state.

P7.5 — Implement guardrails

- Defend against external prompt injection.
- No unrestricted tool execution.

P7.6 — Implement failure handling

- Provider failure →retry/degraded/escalate, never unsafe PASS.

Tests

- Mock LLM tests
- Schema/malformed output tests
- Prompt injection tests
- Context leakage tests
- Provider timeout/retry tests
- Authority-boundary tests

Exit Gate

Pass when AI produces bounded proposals from authorized context but cannot verify, approve, promote or create trusted state.

Deliverables

- AI gateway
- Provider adapters
- Context builder
- Prompt/schema registry
- Guardrails
- AI evidence

Risks / Escalations

- Injection
- AI output treated as truth
- Vendor lock-in
- Uncontrolled cost/latency

P8 — Remediation & DAIL Verification

Objective: Implement the deterministic DAIL mechanism with fixed patches first, then feed validated AI proposals through the same boundary.

Entry: P6 gate passed. P7 contract may be available; fixed-patch work starts first.

Workstreams

ID	Workstream	Scope
P8.1	Candidate State Builder	Only DAIL component combining trusted state + patch.
P8.2	Trusted State	Versioned trusted Terraform state.
P8.3	Invariant Store	Protected security/functional obligations.
P8.4	Verification	PASS/FAIL/UNKNOWN + evidence.
P8.5	Decision	PROMOTE/REJECT/RETRY/ESCALATE.
P8.6	Feedback	Structured rejection evidence/bounded retry.
P8.7	Research Evidence	Full run-to-decision chain.

Milestones

ID	Outcome	Evidence
P8-M1	Trusted baseline	Trusted fixture + verified obligations load.
P8-M2	Unsafe candidate	Canonical regression is rejected.
P8-M3	Safe candidate	Canonical safe candidate is promoted in deterministic simulation.
P8-M4	AI path	Validated AI patch enters identical DAIL path.
P8-G	DAIL gate	Fixed-patch end-to-end DAIL passes.

Implementation Tasks

P8.1 — Build Candidate State Builder

- Apply bounded patch operations to trusted state.
- Reject malformed/unsupported operations.

P8.2 — Build trusted-state manager

- Immutable historical versions + current trusted pointer.
- Rejected candidate never replaces base.

P8.3 — Build invariant store

- Predicate, scope, version, last verified state/evidence.

P8.4 — Build verifier

- Explicit PASS/FAIL/UNKNOWN + evidence/provenance.
- Do not silently convert uncertainty.

P8.5 — Bind verification to impact

- Verify remediation requirement and affected protected obligations.

P8.6 — Build decision controller

- No UI/LLM override.
- Exact candidate version and context binding.

P8.7 — Implement canonical demonstration

- Initial: public SSH violated; app-RDS:5432 verified.
- Candidate 1 fixes SSH but breaks RDS →REJECT.
- Candidate 2 fixes SSH and preserves RDS →PROMOTE.

P8.8 — Implement evidence chain

- Run →patch →candidate →identity →dependency →impact →verification →decision →ground truth/metric references.
- Never overwrite raw evidence.

Tests

- Unit tests for every engine
- Lifecycle tests
- Canonical unsafe/safe scenarios
- UNKNOWN/unsupported tests
- Created-resource baseline tests
- Identity/dependency tests
- E2E reject/promote tests
- Evidence replay tests
- Adversarial bypass tests

Exit Gate

Pass only when fixed-patch DAIL rejects the canonical regression, promotes the safe candidate, preserves evidence, and blocks known bypasses.

Deliverables

- DAIL core
- Trusted/invariant store
- Candidate builder
- Verifier
- Decision controller
- Canonical fixtures
- Evidence pipeline

Risks / Escalations

- Verifier exact semantics may need audit
- Circular oracle
- Candidate privilege leakage
- Incomplete evidence

P9 — Governance & Cloud Promotion

Objective: Connect verified DAIL decisions to a bounded AWS mutation path with explicit governance, preflight and reconciliation.

Entry: P8 gate passed; mutation permissions reviewed.

Workstreams

ID	Workstream	Scope
P9.1	Governance	Approval and policy model.
P9.2	Promotion Controller	Only trusted-state mutation authority.
P9.3	Change Command	Bounded AWS mutation interface.
P9.4	Preflight	Re-check candidate, impact, verification, policy and current state.
P9.5	Execution/Reconciliation	Success/failure/ambiguous outcomes.
P9.6	Evidence	Authorization, command, outcome and post-change observation.

Milestones

ID	Outcome	Evidence
P9-M1	Governance	Approval cannot be bypassed.
P9-M2	Promotion identity	Dedicated least-privilege identity works.
P9-M3	Mutation	One supported remediation executes safely.
P9-G	Promotion gate	Trusted state updates only after authoritative post-change observation.

Implementation Tasks

P9.1 — Implement approval

- Bind to exact candidate version/context.
- Invalidate on relevant changes.

P9.2 — Implement Promotion Controller

- No other service writes trusted state.

P9.3 — Implement bounded mutation

- Start with one/few explicitly supported operations.
- No arbitrary shell or unrestricted apply authority.

P9.4 — Implement preflight

- Fresh observation, candidate, policy, impact and verification checks.

P9.5 — Implement reconciliation

- Timeout/ambiguous external outcome →reconciliation, not trust.

P9.6 — Implement post-change observation

- Update trusted state only after authoritative verification.

Tests

- IAM tests
- Promotion bypass tests
- Stale approval tests
- Idempotency tests
- Failure/timeout tests
- Post-change observation tests
- Atomicity tests

Exit Gate

Pass when one supported AWS remediation completes the governed loop and only Promotion Controller can mutate trusted state.

Deliverables

- Governance model
- Promotion Controller
- Execution IAM
- Mutation interface
- Reconciliation worker
- Evidence

Risks / Escalations

- Overprivilege

- Ambiguous success
- Stale approval

P10 — Trusted State & Continuous Control

Objective: Maintain trusted state over repeated discovery, external drift and remediation iterations.

Entry: P9 gate passed.

Workstreams

ID	Workstream	Scope
P10.1	Reconciliation	Trusted vs authoritative observation.
P10.2	Drift	Relevant drift classification.
P10.3	Reverification	Invalidate/re-establish affected obligations.
P10.4	Continuous Jobs	Scheduled discovery/security/reconciliation.
P10.5	Recovery	Worker/provider failure recovery.
P10.6	Status Contract	Current/stale/invalidated/unknown/degraded semantics.

Milestones

ID	Outcome	Evidence
P10-M1	Drift	External change detected.
P10-M2	Reverification	Affected obligations re-established.
P10-M3	Continuous loop	Repeated runs are idempotent.
P10-G	Trust gate	Relevant contradictory evidence cannot leave trusted state silently valid.

Implementation Tasks

P10.1 — Build reconciliation

- Compare trusted/current authoritative observations.
- Record provenance.

P10.2 — Build drift classifier

- Separate relevant from benign changes.
- Unknown remains visible.

P10.3 — Build invalidation/reverification

- Do not rewrite historical evidence.
- Reverify affected obligations.

P10.4 — Schedule continuous jobs

- Durable checkpoints and overlap protection.

P10.5 — Implement recovery

- Crash/restart resumes safely; never infer trust from crash state.

Tests

- Drift tests
- Idempotent reconciliation
- Worker crash/restart
- Stale evidence
- Unauthorized external change
- Recovery tests

Exit Gate

Pass when the system continuously reconciles trusted state and never silently preserves trust after relevant drift/invalidation.

Deliverables

- Drift/reconciliation
- Continuous jobs
- Reverification
- Trusted-state status APIs

Risks / Escalations

- Eventual consistency
- Evidence invalidation gaps
- Duplicate continuous jobs

P11 — Product Experience & Integrations

Objective: Expose the validated control loop through the UI while preserving backend authority and uncertainty semantics.

Entry: P8–P10 stable; API contracts mature.

Workstreams

ID	Workstream	Scope
P11.1	Shell	Navigation/session/scope.
P11.2	Inventory	Connections/assets/snapshots/freshness.
P11.3	Security	Findings/risk/evidence.
P11.4	Impact	Dependency graph/blast radius/confidence.
P11.5	AI	Explain/recommend as untrusted proposal.
P11.6	DAIL	Candidate/impact/verification/approval/promotion.
P11.7	Trusted/Drift	Trusted, invalidated and drift states.
P11.8	Jobs/Evidence	Async status/audit/evidence.
P11.9	Integrations	Only selected high-value MVP integrations.

Milestones

ID	Outcome	Evidence
P11-M1	Golden path	Connect →discover →finding →AI →candidate →impact →verify →approve →promote →trusted.
P11-M2	Safety UX	Unknown/stale/unsafe states are unmistakable.
P11-M3	Evidence UX	Decision provenance inspectable.
P11-G	Product gate	Complete loop is operable without bypass.

Implementation Tasks

P11.1 — Build shell

- Authz-aware routing; server remains authoritative.

P11.2 — Build inventory/findings

- Show freshness and partial/unknown states.

P11.3 — Build impact view

- Graph is domain-backed and provenance-aware.

P11.4 — Build AI view

- Clearly label generated content as proposal.

P11.5 — Build candidate/DAIL views

- Show exact version, affected obligations and results.

P11.6 — Build governance/promotion UI

- Explicit confirmation; no client-side status mutation.

P11.7 — Build trusted/drift UI

- Do not represent stale/unknown as clean.

P11.8 — Add integrations selectively

- Only integrations justified by MVP operation/evidence needs.

Tests

- Frontend tests
- API contract tests
- Golden-path E2E
- Authorization tests
- Accessibility tests
- Stale/partial/unknown UX tests
- Promotion confirmation tests

Exit Gate

Pass when the complete validated loop is operable through the product UI with no backend safety bypass.

Deliverables

- Frontend
- Golden-path screens
- Evidence UX
- Jobs/notifications
- Selected integrations

Risks / Escalations

- UI scope creep
- UI hides uncertainty
- Domain logic duplicated in frontend

P12 — Production Hardening & MVP Release

Objective: Validate the MVP against security, performance, reliability, recovery, deployment, research and release criteria.

Entry: All prior phase gates passed.

Workstreams

ID	Workstream	Scope
P12.1	Security	Threat-model controls, IAM, secrets, tenant isolation, supply chain.
P12.2	Performance	Load/stress/soak/capacity.
P12.3	Reliability	Fault injection/backpressure/recovery.
P12.4	DR	Backup/restore/RTO/RPO/reconciliation.
P12.5	Deployment	CI/CD, migrations, rollback/reconciliation.
P12.6	Research Readiness	Independent oracle, A/B/C harness, evidence replay, pilot, protocol freeze.

P12.7	Release Ops	SLOs, dashboards, alerts, runbooks, SecOps.
-------	-------------	---

Milestones

ID	Outcome	Evidence
P12-M1	Full tests	All blocking tests pass.
P12-M2	Resilience	Critical failures recover safely.
P12-M3	Pilot	No protocol-breaking defects.
P12-M4	Protocol freeze	Scenarios, baselines, metrics and analysis frozen.
P12-G	MVP gate	Definition of Done and release acceptance complete.

Implementation Tasks

P12.1 — Run security validation

- SAST, dependency/secret/IaC/container checks and IAM review.
- Validate promotion boundary.

P12.2 — Run performance validation

- API latency, async acceptance, discovery throughput, verification work and queue latency.

P12.3 — Run resilience validation

- AWS throttling, worker crash, queue/DB failure, LLM outage and external failure.

P12.4 — Run DR validation

- Restore evidence/state safely; reconcile authoritative state; restore alone never grants trust.

P12.5 — Validate deployment

- Immutable artifacts, safe migrations, rollback and ambiguous deployment reconciliation.

P12.6 — Validate research

- Independent oracle + A/B/C + reproducible evidence + pilot.
- Freeze protocol before main experiment.

P12.7 — Complete release readiness

- SLOs, dashboards, alerts, runbooks, access review and release package.

Tests

- Full system suite
- Security/adversarial suite
- Performance load/stress/soak
- Fault injection
- DR drills
- Deployment/rollback tests
- Independent oracle validation
- A/B/C reset/repeat tests
- Evidence replay
- Pilot

Exit Gate

Pass only when the MVP satisfies the approved Definition of Done, blocking gates are green, critical failures have safe terminal/reconciliation behavior, and the research protocol is frozen.

Deliverables

- Release candidate
- Security report
- Performance/reliability report
- DR evidence
- Deployment evidence
- Pilot report
- Frozen protocol
- MVP acceptance package

Risks / Escalations

- Tests mistaken for empirical proof
- Post-result protocol changes
- Restore mistaken for trust
- Unvalidated scope expansion

5. Cross-Phase Engineering Obligations

Area	Required throughout
Security	Least privilege, secret isolation, tenant isolation, threat-model controls, adversarial testing.
Testing	Meaningful tests accompany implementation; safety-critical paths receive deterministic regression coverage.
Observability	Logs, metrics, traces, product events, audit and evidence remain distinct.
Evidence	Research-critical transitions remain immutable, linked and replayable.
Data	Versioning, provenance, freshness and scope are explicit.
Infrastructure	Terraform is canonical; environment boundaries are enforced.
AI	Provider-agnostic, structured, guarded, evaluated, non-authoritative.
DAIL	Explicit subsystem boundary; deterministic core first.
Research	Independent oracle + A/B/C protocol before main experiment.
UX	UI reflects backend truth; it cannot create trust or certainty.

6. First Coding Vertical Slice

1. Load canonical Terraform fixture.
2. Normalize supported resources.
3. Create trusted-state version.
4. Register protected invariants with PASS evidence.
5. Apply fixed patch through Candidate State Builder.
6. Run identity and change analysis.
7. Build dependency graph.
8. Calculate affected obligations.
9. Invalidate evidence whose applicability changed.
10. Verify remediation requirement and affected obligations.
11. Reject canonical unsafe candidate.
12. Promote canonical safe candidate in deterministic simulation.
13. Persist complete evidence chain.
14. Prove rejected candidate cannot become trusted base.
15. Replay deterministically.

Canonical demonstration: initial state has public SSH violated and application—RDS:5432 reachability verified. Candidate 1 fixes SSH but breaks RDS reachability →REJECT. Candidate 2 fixes SSH and preserves RDS reachability →PROMOTE. The fixed-patch demonstration must work before live LLM integration.

7. Research Audit / Escalation Matrix

Item	Status	Implementation rule
DAIL research contribution / state-promotion interaction	LOCKED	Do not redefine.
Terraform + AWS MVP scope	LOCKED	Do not silently expand.
Candidate State Builder inside DAIL	LOCKED	Only patch—candidate path.
Promotion Controller authority	LOCKED	Only trusted-state mutation authority.
PASS/FAIL/UNKNOWN contract	LOCKED	Do not weaken semantics.
Exact verifier implementation	PROVISIONAL	Validate; escalate if claims change.
Independent oracle	PROVISIONAL	Must avoid circular evaluation.
Verification-work metric	PROVISIONAL	Freeze before main experiment.
A/B/C prompt/context equivalence	PROVISIONAL	Review before main experiment.
Unsupported/unknown policy	PROVISIONAL	Conservative; escalate if semantics affect claims.

8. Master Definition of Done

- Implemented →Validated →Secure →Tested →Observable →Recoverable →Governed →Evidenced →Accepted.
- Deterministic DAIL fixed-patch loop works before live LLM experimentation.
- Canonical regression is rejected; canonical safe candidate is promoted.
- Rejected/UNKNOWN/UNCERTAIN states cannot silently become trusted.
- Identity/dependency/impact are deterministic for supported cases and conservative under uncertainty.
- Only Promotion Controller can update trusted state.
- AWS mutation is bounded, least-privileged and reconciled.
- Every research-critical transition is traceable to raw evidence and analysis.
- Independent ground truth and A/B/C execution are validated before main experiment.
- All blocking system/security/performance/reliability/DR tests pass.
- Implementation deviations are documented; research-semantic changes are reviewed.
- MVP acceptance is based on the approved Definition of Done, not merely feature completion.

9. Execution Protocol for the Implementation Chat

16. Start at P1; do not skip dependency gates to reach AI or UI faster.
17. For every task: define contract →implement →test →observe →document evidence.
18. At each milestone, report: status, what was built, tests, results, files/artifacts, decisions, deviations, risks, audit needed, next milestone.
19. When a locked decision appears technically impossible/unsafe, stop at that boundary and escalate rather than silently redesigning it.
20. Do not interpret implementation success as empirical research success.
21. Do not optimize verification work before correctness and independent ground truth are validated.
22. Do not delete negative/inconvenient research results.

10. Final Directive

This Master Implementation Breakdown is the execution bridge from the completed documentation set to actual implementation. The next actionable step is P1. The project should build the smallest deterministic DAIL core first, validate it thoroughly, connect the LLM second, then build the research evidence/experiment pipeline, governed AWS promotion, continuous trusted-state control, product UX and final hardening.

The objective is not merely a working application. The objective is a reproducible, deterministic, safe, auditable and experimentally defensible implementation of the approved CloudShield AI + DAIL design.